

Reviewer Pack — Noncommutative Fourier Norm Inverse Proportional to Disjoint...

Entry 250c806b0e11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 16:11:54 UTC

Noncommutative Fourier Norm Inverse Proportional to Disjointness Communication Complexity

Entry ID: 250c806b0e11 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 16:11:54 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a $|X| \times |Y|$ matrix M defining a DISJOINTNESS instance, let $\|F(M)\|_{\text{op}}$ denote the operator norm of its noncommutative Fourier transform over the free group F_2 . Then $\|F(M)\|_{\text{op}} = \Theta(1/n)$ for all $n \geq 2$, with constants depending on the group representation parameters.

Rationale (proposer's reasoning):

Noncommutative harmonic analysis provides tools to decompose matrix-valued functions on non-abelian groups. By associating DISJOINTNESS matrices with representations of F_2 , we can leverage Fourier-analytic bounds to derive communication complexity lower bounds via operator norms.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eee49155abb015ae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

# Helper functions for matrix operations
def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(p):
                C[i][j] += A[i][l] * B[l][j]
    return C

def matrix_addition(A, B):
    m = len(A)
    n = len(A[0])
    C = [[A[i][j] + B[i][j] for j in range(n)] for i in range(m)]
    return C

def identity_matrix(n):
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    return I

def transpose(A):
    m = len(A)
    n = len(A[0])
    B = [[A[j][i] for j in range(m)] for i in range(n)]
    return B

# Helper function to generate a random DISJOINTNESS matrix
def generate_disjointness_matrix(n):
    X = set(random.sample(range(2*n), n))
    Y = set(random.sample(range(2*n, 4*n), n))
    M = [[0] * (2*n) for _ in range(2*n)]
    for i in X:
        for j in Y:
```

```

        if i < j:
            M[i][j] = 1
    return M

# Noncommutative Fourier transform using the regular representation of F_2
def noncommutative_fourier_transform(M, rep):
    n = len(M)
    m = len(rep)
    fourier_rep = [[0] * m for _ in range(m)]
    for i in range(m):
        for j in range(m):
            if i == j:
                fourier_rep[i][j] = 1
            else:
                fourier_rep[i][j] = -1
    return matrix_multiplication(fourier_rep, matrix_multiplication(M, fourier_rep))

# Run a single trial with the given seed
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0.0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        M = generate_disjointness_matrix(n)
        F_M = noncommutative_fourier_transform(M, rep)
        operator_norm = max([sum(abs(x) for x in row) for row in F_M])
        total_metric_value += operator_norm
        instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    if abs(mean_metric_value - 0.5) > 0.1:
        conjecture_holds = False
        counterexample = "mean_metric_value deviates from expected"

    return {
        "metric_name": "operator_norm",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mean_metric_value deviates from expected\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67f40197.py", line 104, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67f40197.py", line 80, in run_trial
    M = generate_disjointness_matrix(n)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67f40197.py", line 54, in generate_disjointness_matrix
    M[i][j] = 1
    ~~~^
IndexError: list assignment index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to index error, preventing data collection. Pre-registered support condition cannot be evaluated. | next: Fix the matrix generation code to handle index ranges properly and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112201
2	propose	ollama_remote	qwen3:8b	0	0	35128
3	preregistration	ollama_remote	qwen3:8b	0	0	24367
4	novelty	ollama_remote	qwen3:8b	0	0	20769

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	qwen3:8b	0	0	14337
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15801
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11253
8	judge	ollama_remote	qwen3:8b	0	0	20984

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 254840 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/250c806b0e11.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/250c806b0e11.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/250c806b0e11.tar.gz (if generated)